

NOTES

Multithreading, Multiprocessing, & Concurrency

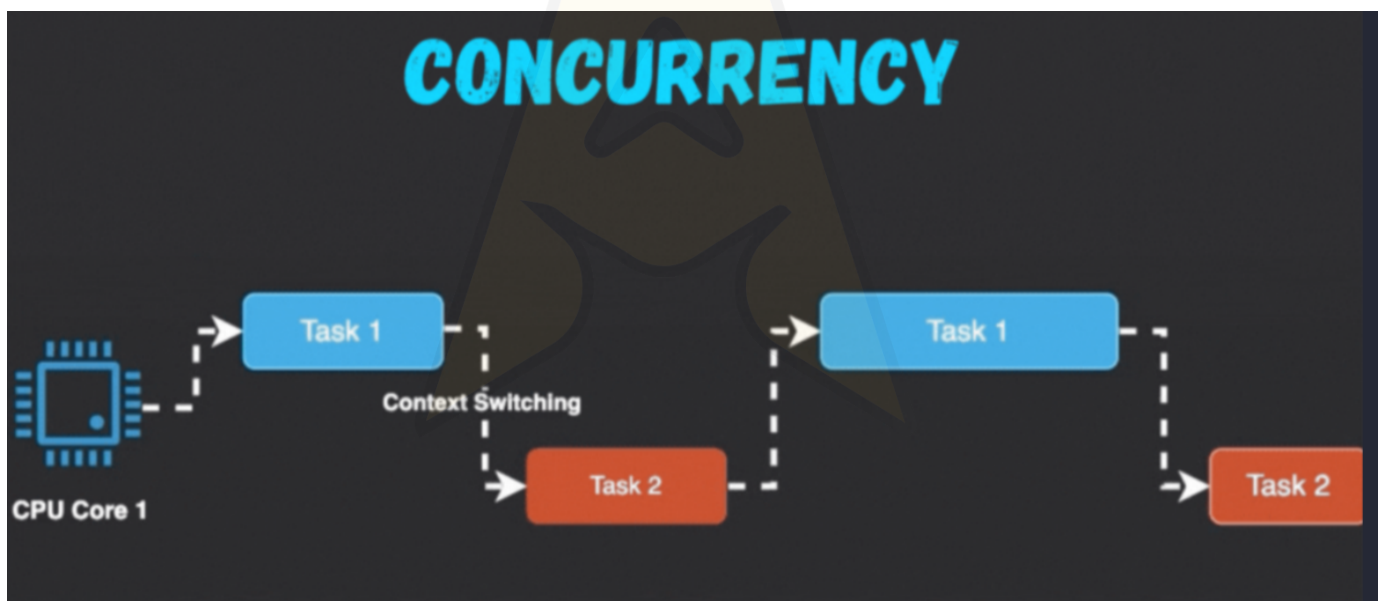


Multithreading, Multiprocessing, and Concurrency

✓ 1. Introduction to Concurrency

Concurrency allows multiple tasks to be in progress at the same time, increasing the efficiency of programs that would otherwise need to wait on I/O operations or computational tasks. In Python, the two main ways to achieve concurrency are:

1. **Multithreading**: Running multiple threads (lightweight processes) within the same process.
 2. **Multiprocessing**: Running multiple processes, each with its own memory space. Each has its strengths and weaknesses:
- Multithreading is ideal for I/O-bound tasks (like file I/O or network requests).
 - Multiprocessing is better for CPU-bound tasks (like mathematical computations).



✓ 2. Multithreading in Python

- **Thread** : A thread (smaller part) is like a lightweight, mini-process that can run alongside other threads within the same application.

Theory of Multithreading

- **Multithreading** allows a program to execute multiple threads concurrently.
- A thread is the smallest unit of a process that can be scheduled by the operating system.
- The Python `threading` module is commonly used for multithreading tasks.

Code Implementation

- Example of multithreading. We'll create a simple program that spawns multiple threads to perform a CPU-light task, like counting.

```
import threading
import time

# Function to simulate a task
def task(name, count):
    for i in range(count):
        print(f"{name} is running iteration {i}")
        time.sleep(0.5)

# Create threads for multithreading
thread1 = threading.Thread(target=task, args=("Thread 1", 5))
thread2 = threading.Thread(target=task, args=("Thread 2", 5))

# Start the threads
print("Starting threads")
thread1.start()
thread2.start()

# Wait for both threads to complete
thread1.join()
thread2.join()
print("All threads completed.")
```



```
Starting threads
Thread 1 is running iteration 0
Thread 2 is running iteration 0
Thread 1 is running iteration 1
Thread 2 is running iteration 1
Thread 1 is running iteration 2
Thread 2 is running iteration 2
Thread 1 is running iteration 3
Thread 2 is running iteration 3
Thread 1 is running iteration 4
Thread 2 is running iteration 4
All threads completed.
```

Explanation of the Code:

- We define a task function, which each thread will run, simulating a task by printing output and pausing briefly.
- thread1 and thread2 are created and run concurrently by calling .start() on each.
- The join() method waits for both threads to complete before moving on to the next statement, ensuring the main program waits until both threads are done.
- The program will output interleaved messages from both threads, indicating that they are running concurrently.

✓ 3. Multiprocessing in Python

Theory of Multiprocessing

- Multiprocessing involves running multiple independent processes. Each process has its own Python interpreter and memory space, allowing it to bypass the GIL limitation, making it suitable for CPU-bound tasks.
- The multiprocessing module provides an interface for spawning processes, passing data between them, and synchronizing.

Code Implementation

- This example demonstrates how to use multiprocessing to perform a CPU-heavy task, like calculating squares of numbers, using multiple processes.

```
from multiprocessing import Process
import time

# Function to perform a CPU-bound task
def calculate_square(number):
    for _ in range(10000000):
        number * number
    print(f"Square calculated for {number}")

if __name__ == "__main__":
    # Set up two processes
    process1 = Process(target=calculate_square, args=(10,))
    process2 = Process(target=calculate_square, args=(20,))

    print("Starting processes")
    # Start the processes
    process1.start()
    process2.start()

    # Wait for both processes to complete
    process1.join()
    process2.join()
```

```
print("All processes completed.")
```

Explanation of the Code:

- calculate_square function simulates a CPU-bound task by performing repetitive multiplications.
- process1 and process2 are created and started with different numbers as arguments.
- Both processes run concurrently in their own memory spaces. The .join() ensures that the main program waits for both processes to finish.
- The program will output the completion messages from each process after they finish the CPU-bound task.

✓ 4. Comparing Multithreading and Multiprocessing

- Let's consider the following table to help decide when to use multithreading vs. multiprocessing:

Multithreading	Multiprocessing
Multithreading is creating and starting threads to execute a task within a single process.	Multiprocessing is executing multiple processes within a multi-core CPU.
Multithreading costs less to set up and use.	Multiprocessing costs more to set up and use.
Multithreading is handled by an interpreter. In program languages like Python, it is handled by a Cpython interpreter.	Multiprocessing is generally handled by an Operating System.
Multithreading works dependently so it has a shared memory space.	Multiprocessing works independently so it has separate memory space.
Multithreading handles input/output tasks such as database queries, web server requests, or any data processing that involves shared data.	Multiprocessing handles CPU tasks such as high-level computing, scientific computing, etc.
Multithreading is less complex because it runs in the same process.	Multiprocessing is more complex because it runs on separate processes.

✓ 5. Real world Examples of Multithreading , Multiprocessing and Concurrency.

1. Multithreading :

- Multithreading is ideal for I/O-bound tasks where tasks spend a lot of time waiting for external resources, such as file operations, network requests, or user inputs. In multithreading, multiple threads run within a single process, sharing the same memory space and making communication between threads easy.

Real-World Examples of Multithreading:

- **Web Browsers:**

Web browsers use multithreading to handle multiple tabs or simultaneous page loads. When a user opens several tabs, each one loads content and images, which are I/O-bound tasks. Each tab runs in its own thread, allowing content in one tab to load while the user interacts with another.

- **Downloading Files:**

Download managers often use multithreading to split large files into chunks and download each chunk in parallel. Each thread is responsible for downloading a portion of the file, which improves download speed.

2. Multiprocessing :

- Multiprocessing is best suited for CPU-bound tasks that require significant computing power, like data processing, simulations, or complex calculations. By using separate processes, multiprocessing can take advantage of multiple CPU cores, which is ideal for tasks that don't need to share memory frequently.

Real-World Examples of Multiprocessing:

Machine Learning Training:

- Training machine learning models, especially deep learning models, is highly CPU- or GPU-bound. Multiprocessing allows each process to handle a subset of the data, speeding up training by allowing multiple cores to work on different parts of the training set simultaneously.

Video Rendering and Processing:

- Video editing software often uses multiprocessing to handle video rendering. Each process can be assigned a section of the video to render in parallel, taking advantage of multiple CPU cores to speed up the process significantly.

3. Concurrency (Multithreading and Multiprocessing Together):

- Concurrency allows multiple tasks to make progress over the same period of time. Whether through multithreading (for I/O-bound tasks) or multiprocessing (for CPU-bound

tasks), concurrent programming is useful in applications where you need to handle a mix of both types of tasks.

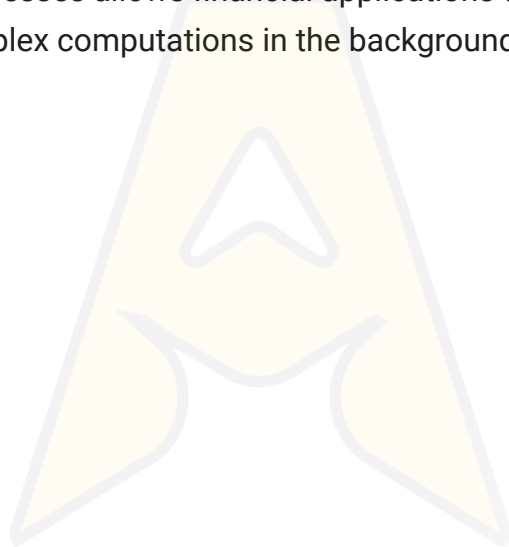
Real-World Examples of Concurrency:

Data Pipelines:

- In data engineering, data pipelines often involve a combination of data retrieval (I/O-bound) and data processing (CPU-bound). For example, a pipeline might fetch data from an API (handled by threads) and then process and analyze the data using multiple CPU cores (handled by processes), thus achieving concurrent data flow and transformation.

Financial Systems:

- In finance, systems that calculate real-time stock prices or analyze market data handle both I/O and CPU tasks. Data needs to be fetched and displayed (I/O-bound), while market predictions and risk calculations are CPU-intensive tasks. Combining threads and processes allows financial applications to keep the UI responsive while performing complex computations in the background.



THANK YOU



Stay updated with us!



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)